

MorningBriefing.AI

Final Project Report

Course: CSE 416 Software Engineering

Team: Ahn Minsoo · Choo Sungmin · Kwon Dahyun

Submitted: June 10, 2026

Live URL: <https://morningbriefing.dksalstn0621.workers.dev>

Repository: https://github.com/MinsooAhn02/CSE_416_3pip

Table of Contents

- 1. Project Overview**
 - 1.1 Problem Statement
 - 1.2 Solution
 - 1.3 Core Value Proposition
 - 1.4 Core Features
- 2. Team**
- 3. Technology Stack**
 - 3.1 Frontend
 - 3.2 Backend and Infrastructure
 - 3.3 External APIs
- 4. Architecture Overview**
 - 4.1 High-Level Architecture
 - 4.2 State Management
 - 4.3 Caching Strategy
 - 4.4 Data Fetch Pipeline
- 5. Feature Implementation**
 - 5.1 AI Briefing
 - 5.2 Smart Widgets
 - 5.3 Auto Diary Writing
 - 5.4 Personalization Engine
 - 5.5 Google Integrations
 - 5.6 Internationalization
- 6. Requirements & Use Case Coverage**
- 7. Database Schema**
- 8. Security Design**
- 9. Development Process**
 - 9.1 Sprint Summary
 - 9.2 Schedule Changes
 - 9.3 Team Contributions
 - 9.4 Quality Process
- 10. API Design**
- 11. Testing**
- 12. Known Issues and Limitations**
- 13. Reflection and Future Work**

1. Project Overview

MorningBriefing.AI is a browser new-tab dashboard that turns the first moment of a user's day into a personalized, AI-curated briefing, delivered automatically every time a new tab is opened.

1.1 Problem Statement

Busy professionals and students check multiple apps every morning to get the information they need: news, weather, stock prices, calendar events. Switching between these sources is slow, and most of what they read is not relevant to them. The result is information fatigue: users spend time reading but absorb little.

Existing solutions either aggregate data without context (plain RSS readers, weather apps) or require heavy configuration (home automation dashboards). No single product delivers a personalized, narrative-style briefing that adapts to what the user actually cares about.

1.2 Solution

Morning Briefing.AI is a desktop web app that helps people get off to a more efficient start to their day. Every morning, it offers customized AI-based briefings that summarize the weather, stock market data, the latest news, and your Google calendar in one simple summary. Instead of checking multiple apps before 9 a.m., you now only need to open one tab. The dashboard assembles data from eight sources in parallel, caches aggressively to minimize load time, and presents everything in a three-column layout. An AI briefing at the top of the screen narrates the day in plain language. The homepage is self-explanatory for first-time visitors: the login page itself explains what the product does and how to start, so a stranger can understand and use the site on their own.

Live URL: <https://morningbriefing.dksalstn0621.workers.dev>

Repository: https://github.com/MinsooAhn02/CSE_416_3pip

1.3 Core Value Proposition

| “Turn information into clarity. Turn your day into a story.”

1.4 Core Features

#	Feature	Description
1	AI Briefing	A structured daily summary built from deterministic JS sections plus two narrow Groq calls. Temperature 0.1 ensures consistent output. Saved as 3-hour snapshots for diary generation.

2	Smart Widgets	User-defined keywords. Groq classifies each into one of 17 categories. Tavily fetches category-appropriate articles. The results surface in both the widget and the briefing.
3	Auto Diary Writing	A daily diary generated from briefing snapshots, Q&A answers, and the previous day's entry. PIN-protected. User can dislike a draft and get a Groq-rewritten version.

2. Team

MorningBriefing.AI was built by three Stony Brook University students for CSE 416 (Software Engineering), Spring 2026. Sprint management used Jira:

Scrum Board:

<https://stonybrook-team-3pip.atlassian.net/jira/software/projects/SCRUM/boards/1>

Member	Role	Email
Ahn Minsoo	Infrastructure, DevOps & Backend Developer	minsoo.ahn@stonybrook.edu
Choo Sungmin	Backend Developer & UI/UX Designer	sungmin.choo@stonybrook.edu
Kwon Dahyun	Frontend Developer & UI/UX Designer	dahyun.kwon@stonybrook.edu

Each member's primary responsibilities and beyond-scope contributions are detailed in Section 9.3.

3. Technology Stack

We chose each technology based on a concrete requirement. This section covers what we use and why.

3.1 Frontend

Library / Tool	Version	Role and Rationale
TypeScript	—	Full codebase migration completed 2026-05-24 (commit 365b392, ~365 files). strict:false reduces migration effort without blocking the build.
React	18.3	UI framework. React.lazy + Suspense on all 9 widgets and modals for code splitting. React.memo on each widget prevents unnecessary re-renders.
Vite	6	Build tool. VITE_* env vars are injected at build time, which is why re-build is required before each deploy.
Tailwind CSS	3.4	Utility-first styling. All layout, color, and spacing is Tailwind class-based with no CSS files except global resets.
Zustand	5	State management. 10 domain-specific stores. Each store owns one domain; cross-store dependencies are explicit.
framer-motion	—	Animations for panel open/close and widget transitions.
@hello-pangea/dnd	—	Drag-and-drop for stock symbol reordering in the Stocks widget modal.
i18next / react-i18next	—	Korean and English support. Language toggle in TopNav. Language-change event triggers re-fetch of news, trends, and Smart Widget data.

3.2 Backend and Infrastructure

Service	Role and Rationale
Supabase	Backend-as-a-Service: Google OAuth, PostgreSQL with Row-Level Security, and 9 serverless Edge Functions (Deno runtime). Supabase removed the need to stand up a separate API server, which was the right call for a three-person team on a semester timeline.
Groq LLM	AI inference for briefing generation, diary generation, diary rewrites, and keyword extraction. Chosen for near-real-time inference speed. At temperature=0.1, output is consistent and predictable.
Cloudflare Workers	Static SPA hosting. wrangler.jsonc's not_found_handling: "single-page-application" routes all paths to index.html, which is required for Google OAuth callbacks (/?code=xxx) to work correctly after redirect.

3.3 External APIs

API	Purpose
OpenWeatherMap	Current weather data (temperature, humidity, precipitation) and Air Pollution API for AQI. Called by the weather Edge Function.
Twelve Data + Stooq fallback	Stock price quotes. Primary: TwelveData (returns type and currency metadata). Fallback chain: Yahoo Finance, then Stooq CSV, then open.er-api.com for currency pairs.
Tavily	LLM-optimized news and trend search. Three modes: news, trends, and generic search. Queries batched to reduce API cost: N Smart Widget section calls reduced to 1-2 calls per keyword.
Google Calendar	Events and Tasks CRUD. OAuth-gated. Token refresh handled by the google-refresh Edge Function; expired tokens recover without re-login.
Google Fitness	Steps, sleep duration, calories, and heart rate from Google Fit API. Fitness scope requires an annual CASA Tier 2 assessment for full GCP verification.
Google Maps / Places	Location autocomplete for the weather widget city input (GooglePlacesLocationField component).

4. Architecture Overview

The system has three tiers. The React SPA runs entirely in the browser. All third-party API secrets live in Supabase Edge Functions — the browser never sees them. Supabase handles auth, the database, and the serverless API layer.

4.1 High-Level Architecture

Browser (React SPA) - Zustand stores (single source of truth per domain) - React.lazy widgets (code-split, on-demand) - Service layer calls Edge Functions	Supabase (BaaS) - Auth: Google OAuth - PostgreSQL + RLS (9 tables) - Edge Functions (Deno): secret-holding proxy
External APIs (all behind Edge Functions — secrets never in browser) Google Calendar / Tasks / Fit OpenWeatherMap TwelveData / Yahoo / Stooq Tavily Search Groq LLM	

Every call to an external API goes through an Edge Function. The client bundle contains only the Supabase anon key — a public, RLS-scoped credential — and never any third-party API keys.

4.2 State Management

We use 10 Zustand stores, one per domain. This is the team's documented code convention: stores are bounded to a single concern. Stores do not call each other arbitrarily — cross-store dependencies are listed explicitly in DOCS.md §5.

Store	Persists To	Owns
useAuthStore	localStorage + Supabase Auth	Login state, user object, providerToken (memory only)
useSettingsStore	localStorage + user_settings	Theme, tone, clock format, stock symbol list, PIN mode
useDataStore	localStorage + api_cache	Weather, stocks, news, trends, health, briefing data
useWidgetStore	localStorage + widget_layouts	Dashboard layout, widget visibility, Smart Widget data, font size
useDiaryStore	localStorage + diaries	Diary entries, Q&A answers, PIN state
useGoogleCalendarStore	Memory + local fallback	Calendar events and Tasks (Google Calendar API)

useBriefingHistoryStore	localStorage + briefing_snapshots	Time-stamped briefing snapshots (3-hour save interval)
useTodoStore	localStorage + todos	Daily todo list (merged from Google Tasks)
useOnboardingStore	Memory (session only)	Multi-step onboarding flow state. Extracted from useAuthStore on 2026-05-24 for clearer ownership.
useQuickLinksStore	localStorage	User-defined quick links (Naver, YouTube, Instagram, Google by default)

The useOnboardingStore extraction is a concrete example of modular refactoring. Before extraction, onboarding state was tangled with auth state in useAuthStore, making both harder to reason about. After extraction, OnboardingModal.tsx reads exactly one store.

4.3 Caching Strategy

We use a three-layer cache: memory, localStorage, and Supabase api_cache table. The cache-first design means a returning user sees their last data immediately without waiting for a network round trip. Fresh data loads in the background and replaces the stale version once it arrives.

Layer	Scope	Details
Layer 1: Memory	Session lifetime	Geolocation cache (5 min). Trends in-memory map (_trendsMemCache) checked before DB read.
Layer 2: localStorage	Cross-session	mb_cache_{key} pairs with mb_cache_{key}_at timestamps. Fallback when Supabase is unavailable.
Layer 3: Supabase api_cache	Cross-device	6-hour TTL. Primary key is a semantic string (e.g., u:{userId}:news_KR_ko_base) enabling deterministic upsert without a prior SELECT. TTL extended from 1h to 6h (commit 86a34a8) to cut Tavily API cost by ~6x.

Cache Key Design

Cache keys encode exactly what makes a response unique. The news key includes language and interest fingerprint so two users with different interests get separate cache entries, and switching language does not invalidate the other-language cache.

Data Type	Cache Key Pattern
Weather	weather_{roundedLat}_{roundedLon}
Stocks	stocks_{sym1}_{sym2}...
Trends	trends_full_{lang}

News	news_{lang}_{interestFingerprint}
Smart Widget	\${keyword}_\${lang} (e.g., camera_en, camera_ko)
Calendar (today)	calendar_today

interestFingerprint = top-5 interest keywords joined with "+", or "base" if none are set. Changing a keyword changes the fingerprint, which generates a new cache entry for the personalized query while leaving base queries intact.

4.4 Data Fetch Pipeline

fetchAll() is the central fetch orchestrator. It runs on login, tab re-focus, and a 5-minute poll. Each call checks the 6-hour TTL before making any network request.

1. Compute cache key for each data type
2. If force=false, check readApiCache (DB layer, 6-hour TTL)
3. Cache hit: update Zustand state, return immediately
4. Cache miss: POST to Edge Function (25-second timeout)
5. Success: normalize response, write to api_cache (upsert), update state, apiStatus="ok"
6. Failure: set apiStatus="error", fall back to mock data

An initPhaseRef guard prevents React StrictMode's double-mount from firing fetchAll twice on the same login. Without this guard, the app would make duplicate API calls on every fresh load.

5. Feature Implementation

This section covers the six major features with the technical decisions behind each.

5.1 AI Briefing

The AI Briefing is the centerpiece of the dashboard. It generates a structured daily summary every hour. The key design decision is the separation of structure from content: JS controls the section skeleton; Groq fills only two slots.

Deterministic Sections + Narrow AI Pattern

A common mistake in AI-generated dashboards is sending the entire prompt to the LLM and hoping for a consistent structure. We did not do that. `BriefingWidget.tsx` calls `aiService.generateDetailedBriefing()`, which builds four sections entirely in JS:

Section ID	Name	Construction
<code>header</code>	Header	Date and weather summary assembled in JS. No AI call.
<code>schedule</code>	Schedule	Today's and tomorrow's calendar events filtered and sorted in JS. Before noon: today's events. After noon: remaining today plus tomorrow.
<code>yesterday</code>	Yesterday	Yesterday's diary entry rewritten to 1-2 past-tense sentences by Groq (call #1, temperature=0.1).
<code>latest_info</code>	Latest Info	Three sub-blocks: <code>latest_smart</code> (JS, from Smart Widget data), <code>latest_news</code> (top 3 articles with 1-sentence AI summary — Groq call #2), <code>latest_trends</code> (top 3 trends). The two Groq calls run in parallel.

Two parallel Groq calls replace what would otherwise require one large, unpredictable prompt. Temperature 0.1 keeps output consistent across sessions.

The briefing regenerates once automatically when `tomorrowEvents` arrives after initial load, guarded by `tomorrowRegenDoneRef`. This handles afternoon users whose initial briefing generates before tomorrow's calendar data has loaded.

Briefing Snapshots

Every 3 hours (or on manual refresh), `useBriefingHistoryStore` saves a snapshot of the current briefing to `localStorage` and to the `briefing_snapshots` Supabase table. Each snapshot includes `capturedAt`, `source` ("auto" or "refresh"), the full text, and a parsed sections array. These snapshots serve one purpose: diary generation.

5.2 Smart Widgets

Smart Widgets let users add any keyword and see a curated content block for that topic. The system classifies keywords, plans what sections to show, and fetches articles accordingly.

17-Category Taxonomy

When a user adds a keyword, Groq classifies it into one of 17 categories: tech, food, person, travel, sports, entertainment, science, health, finance, education, fashion, automotive, gaming, music, art, politics, or general. Each category maps to a different section plan — for example, a "person" keyword triggers biography, recent news, and social media sections, while a "food" keyword triggers recipe, restaurant, and nutrition sections.

Tavily Query Batching

Without batching, an N-section Smart Widget would make N separate Tavily calls. The `searchSmartSectionsBatched()` function collapses these into 1-2 calls by merging section topics into a single query. This change (commit 86a34a8) cut Tavily API usage significantly and triggered the `api_cache` TTL extension from 1h to 6h.

Category Override UI

Groq's auto-classification is accurate but not perfect. If a keyword is misclassified, the user can click the [category emoji + dropdown arrow] button in the widget header to manually correct it. The override is stored in `useWidgetStore` and persists across sessions.

Language-Aware Filtering

In Korean mode, article fetches use `scoreArticleForLanguage()` to filter results. The scoring function assigns a high score to articles with Hangul in the title, a medium score for Hangul in the content, and rejects articles with no Korean content. This produces cleaner Korean results without blocking the entire domain list.

5.3 Auto Diary Writing

The diary system generates a daily narrative entry automatically, protects it with a PIN, and lets users provide feedback to improve the output.

Midnight Trigger and Missed-Day Recovery

`useMidnightTrigger.ts` runs once on the first login of each day. It checks the date of the last diary entry and compares it to today's date. For each day in the gap, it checks whether briefing snapshots exist for that date — if they do, it infers the user accessed the app that day and generates a diary entry. Days with no snapshots are skipped.

This design avoids generating fake diary entries for days the user did not use the app. It also handles the case where a user skips a weekend and comes back Monday: diaries for both Saturday and Sunday (if the app was used) are generated automatically on Monday's first login.

Generation Context

`diaryGenerationService.buildDiaryGenerationContext()` assembles up to 6 briefing snapshots from the target date, extracts 8 headline lines from the sections array, pulls in Q&A answer pairs for that day, and includes the previous day's diary entry. This context is passed to `aiService.generateDiary()` as a single Groq prompt.

Feedback Loop

The diary feedback model has four states: no feedback, liked, disliked (feedback pending rewrite), and confirmed rewrite. The baseline AI draft is stored as `aiGeneratedDiary` and is never overwritten — it is the immutable reference. Feedback accumulates in the history array with timestamps. `confirmRewrite()` is irreversible: it overwrites diary and syncs to the diaries table.

PIN Protection

The diary PIN is hashed in the browser using the Web Crypto API (SHA-256) before storage. The hash is never compared on the server — verification happens client-side. The raw PIN is never stored anywhere. `setPIN()` and `verifyPIN()` in `useDiaryStore` handle all PIN operations.

5.4 Personalization Engine

The personalization system combines static interest selection with dynamic keyword learning driven by user behavior.

Fixed Interests (Static Layer)

During onboarding, users select interest categories from 8 options: news, tech, fashion, finance, health, food, entertainment, and sports. These selections are stored as `fixedInterestIds` in `user_settings` and feed directly into news queries and Groq context via `buildPersonaContext()`.

Dynamic Keyword Learning

When a user saves a Q&A answer, `bumpKeyword()` extracts keywords and scores them. When `useMidnightTrigger` fires, `runPersonalizationBatch()` runs Groq keyword extraction on all diary and Q&A text from the previous day, inserts results into `keyword_score_log`, and recalculates `user_settings.keyword_interests`.

30-Day Decay Formula

The score formula penalizes old activity:

```
score = sum(base_weight * (30 - elapsed_days) / 30) where elapsed_days < 30
```

Source weights: personal Q&A = 2, diary = 1. Logs older than 30 days are purged from keyword_score_log. A keyword from three weeks ago contributes less than one from yesterday, keeping the interest profile current without manual cleanup.

News Query Injection

On fetchNews(), the top-5 keywords from keywordInterests (sorted by score descending) are prepended to the Tavily query: "Focus topics: kw1, kw2, ..." in English or "관심 주제: kw1, kw2, ..." in Korean. The joined keyword string becomes the interestFingerprint component of the cache key, so personalized results are cached separately from the base query.

5.5 Google Integrations

Google Calendar, Tasks, and Fitness are all OAuth-gated. The token lifecycle is the hardest part of this integration.

Token Refresh with Auto-Retry

All Google API calls go through invokeGoogleWithAuth(name, body, token). If the response status is 401 or 403 (checked by isGoogleAuthErrorMessage()), the wrapper calls ensureProviderToken(forceRefresh:true), which calls the google-refresh Edge Function to exchange the stored provider_refresh_token for a new access token. The original call then retries once with the new token.

This pattern fixes the "blank calendar on next-day re-entry" bug without forcing re-login. Before this was in place, expired provider_tokens caused the calendar to show an empty month for returning users.

OAuth Token Security

The provider_token lives in memory only inside useAuthStore.providerToken. The secureStorage adapter in src/lib/supabase.ts strips provider_refresh_token from localStorage before it is written. This eliminates a token theft vector: if an attacker injects malicious JS via XSS, they cannot read the refresh token from localStorage because it was never stored there.

5.6 Internationalization

The app supports full Korean and English across all user-visible text, including AI-generated content.

i18next Setup

Language resources are split into ko.json and en.json. The TopNav language toggle calls `i18n.changeLanguage()`, which emits a `languageChanged` event. A module-level listener in `useDataStore.ts` catches this event and re-fetches news and trends with `force=true`.

Per-Language Cache Keys

Because `news_ko_...` and `news_en_...` are separate cache keys, switching language reuses the same-language cache if it is within the 6-hour TTL. A user who switches from Korean to English and back within 6 hours does not pay for a redundant Tavily call.

Korean Article Pipeline

After a Tavily news fetch in Korean mode, `translateArticlesToKorean()` post-processes each article's title and content through Groq. The translated payload is backfilled into `api_cache` so subsequent cache hits return pre-translated text without re-translating on every page reload.

5.7 User Onboarding and Newcomer Guidance

A first-time visitor can land on MorningBriefing.AI, understand what it does, and start using it without any outside help. The login page is built to teach a stranger the product before they sign in, and the post-login flow personalizes the dashboard from the very first visit. This directly satisfies the course requirement that an unfamiliar visitor can operate the site on their own and that any audience member can sign up during the live demo.

Login Page as a Self-Guided Guide

The login page in `src/components/layout/LoginScreen.tsx` doubles as a self-guided newcomer guide. Above the fold, it shows the app name, the Continue with Google button, and a bouncing "New here? Learn how it works" cue that scrolls down to the instructions. Below that sit scrollable instruction cards a visitor can read top to bottom:

What is MorningBriefing.AI? explains that the app gathers weather, stocks, news, trends, calendar, health, and a private diary into one screen, adds an AI-written morning briefing, and runs as both a web app and a Chrome new-tab extension.

How to get started: four numbered steps. Sign in with Google, choose your interests, connect Google Calendar and Fitness (optional and skippable), and explore your dashboard.

What you'll see: an eight-feature grid covering AI Briefing, Weather, Stocks, News and Trends, Calendar and Tasks, Health, Diary and Q&A, and Smart Widgets, each with a one-line description.

Make it yours: drag-and-drop layout, per-widget refresh, Settings (theme, clock style, temperature units, tickers, diary PIN), adding Smart Widgets for any keyword, and an English/Korean toggle.

Google unverified app notice: explains the Testing-mode warning and walks the user through it (Advanced, then Continue to MorningBriefing.AI), and reassures the user that this is a standard policy gate, not a security problem.

Post Sign-In Onboarding Wizard

After sign-in, a two-step onboarding wizard runs in `OnboardingModal`, driven by `useOnboardingStore`. Step 0 selects interest categories. Step 1 grants Calendar and Fit permissions and is skippable. `finishOB()` persists the choices to `user_settings`, so the dashboard is personalized from the first visit.

On the first daily login, a `FirstLoginBriefingModal` greets the user with their first AI briefing. The product's core value is visible immediately, before the user does any setup of their own.

Taken together, these flows mean a first-time visitor can understand and operate the product without external help, and any audience member can sign up and use it live during the final presentation.

6. Requirements and Use Case Coverage

All core use cases from the SRS are implemented and working in the deployed product. One requirement was modified mid-project with an instructor-equivalent replacement; it is marked accordingly below. The six functional-requirement groups in SDD section 2.1 form the backbone, and each of the five use cases in SDD section 2.2 maps to one of those groups.

6.1 Functional Requirement Groups

Each requirement group from SDD section 2.1 maps to a delivered feature. Every group is complete in the deployed product.

SDD 2.1 Requirement Group	Delivered by	Status
2.1.1 Briefing	AI Briefing (<code>BriefingWidget</code> , <code>FirstLoginBriefingModal</code> , <code>groq Edge Function</code>), 3-hour briefing snapshots	Completed
2.1.2 Journaling and Daily Question	Auto Diary Writing and daily Q&A (<code>useDiaryStore</code> , <code>useMidnightTrigger</code>), SHA-256 PIN, dislike-to-rewrite feedback loop	Completed
2.1.3 Calendar and Task Management	Calendar and Tasks widget, Google Calendar and Tasks sync (<code>events/tasks Edge Functions</code> , <code>invokeGoogleWithAuth</code>), <code>todos</code> table	Completed
2.1.4 Widget System and Dashboard	Drag-and-drop widget layout (<code>useWidgetStore</code> , <code>@hello-pangea/dnd</code>), Smart Widgets (17-category Groq classification), weather, stocks, news, trends, and health widgets	Completed
2.1.5 User Settings and Personalization	7-tab <code>SettingsModal</code> , 8 fixed interests, dynamic keyword learning with 30-day decay, full EN/KO i18n	Completed
2.1.6 Data Persistence and Caching	Three-layer cache (memory to <code>localStorage</code> to Supabase <code>api_cache</code> , 6-hour TTL), RLS-isolated persistence	Completed

6.2 Use Case Traceability

The five use cases in SDD section 2.2 are listed below with their status, where each is implemented, and a short note. Every use case is satisfied in the deployed product.

Use Case / Requirement	Status	Where Implemented	Notes
------------------------	--------	-------------------	-------

UC-01: Read the Morning Briefing	Completed	AI Briefing (BriefingWidget, FirstLoginBriefingModal, groq Edge Function)	Maps to 2.1.1. First daily open shows the full briefing modal; the 3-line summary stays in the widget.
UC-02: Respond to the Daily Journaling Question	Completed	Diary and Q&A (useDiaryStore, user_qa table)	Maps to 2.1.2. Answers auto-save on blur and feed the nightly diary synthesis.
UC-03: Manage Tasks and Calendar Events	Completed	Calendar and Tasks widget (events/tasks Edge Functions, invokeGoogleWithAuth)	Maps to 2.1.3. Two-way Google Calendar and Google Tasks sync; completed items flow into the diary.
UC-04: Automatic Midnight Journal Generation	Completed	Midnight synthesis (useMidnightTrigger, diaryGenerationService)	Maps to 2.1.2. Runs on first login each day, with missed-day recovery from briefing snapshots.
UC-05: Review Past Journal Entries	Completed	Diary history modal (useDiaryStore, diaries table)	Maps to 2.1.2. Past entries are read-only; only the current day stays editable.

6.3 Modified Requirement

One requirement changed during the project. The original Desktop History requirement was replaced by Smart Widget keyword learning plus briefing snapshots. Desktop browser history access needs a chrome.history extension permission that does not work with an SPA on Cloudflare Workers, so the replacement collects equivalent personalization signal from in-app activity instead. This change is marked Modified (approved-equivalent) and was documented as SCRUM-24 in SCHEDULE.md and the Milestone 4 update.

Original Requirement	Status	Replacement and Rationale
Desktop History (passive personalization from browser history)	Modified (approved-equivalent)	Replaced by Smart Widget keyword learning and briefing snapshots. Avoids the chrome.history permission, which is incompatible with the Cloudflare Workers SPA, and delivers equivalent

		personalization from in-app activity. (SCRUM-24)
Persona selector in onboarding	Modified (approved-equivalent)	Replaced by 8 fixed interest categories, which feed Tavily news queries and Groq context directly and produce a stronger personalization signal than a single persona label. (SCRUM-16 / SCRUM-25)

No core SRS use case was dropped. The only change was an equivalent replacement of two requirements, both approved and documented.

7. Database Schema

The database uses 9 tables across the base schema and three migrations. All user-scoped tables enforce Row-Level Security at the database layer. RLS is not bypassed by any application-layer check — it is the only enforcement layer.

7.1 Table Reference

Table	Source File	Purpose
user_settings	schema.sql	All per-user preferences: theme, tone, clock, stock symbols, persona, fixed_interests, keyword_interests, onboarding_perms, widget layouts, PIN mode.
widget_layouts	schema.sql	Per-user widget grid positions at three breakpoints (lg, md, sm).
todos	schema.sql	Daily todo items. Columns: is_fixed (persists across days), is_recurring (generated column). Non-fixed todos reset at midnight.
smart_keywords	schema.sql	User-defined Smart Widget keywords. Unique constraint on (user_id, keyword).
api_cache	schema.sql	6-hour TTL external API response cache. Primary key is a semantic string (e.g., u:{userId}:news_KR_ko_base). Semantic PK enables deterministic upsert without a prior SELECT.
diaries	add_personalization.sql	Per-date diary entries: aiGeneratedDiary, editedDiary, notes, memo, Q&A answers, feedback history.
keyword_score_log	add_personalization.sql	Raw keyword scoring events from Q&A and diary text. Rolling 30-day window. Indexed on (user_id, logged_date).
user_qa	add_user_qa.sql	Daily Q&A question/answer pairs. Indexed on (user_id, asked_date).
briefing_snapshots	app-level	Time-stamped briefing snapshots used for diary generation and missed-day recovery.

7.2 Notable Schema Decisions

Semantic Primary Key on api_cache

Most tables use UUID primary keys. api_cache uses a human-readable string key: u:{userId}:news_KR_ko_base. This is intentional. Because the key is deterministic from user ID, language, and interest fingerprint, the application can issue an UPSERT with no prior SELECT. UUID PKs would require a GET-then-INSERT pattern or a UNIQUE constraint on a separate column.

Row-Level Security on All User Tables

Every table that holds user data has four RLS policies: SELECT, INSERT, UPDATE, and DELETE, each scoped to `auth.uid() = user_id` (or `= id` for `user_settings`). This was set up in Sprint 1 (SCRUM-17) and re-verified in Sprint 4 (SCRUM-27). The SQL files include explicit GRANT blocks because Supabase changed its auto-grant behavior for new tables on 2026-05-30.

keyword_interests Stored in user_settings

The `keyword_interests` column is a JSONB array on the `user_settings` row: `[{keyword, category, score}]`. This keeps the aggregated interest profile in one place alongside all other user preferences. The raw events live in `keyword_score_log`. Separating aggregate from log means reads are fast (single row join) while the full history is available for recomputation.

7.3 Trigger

A single trigger `update_updated_at()` auto-updates the `updated_at` timestamp on `user_settings`, `widget_layouts`, and `diaries` on any row update.

8. Security Design

Security decisions were made early and treated as non-negotiable constraints, not afterthoughts.

Security Control	Implementation
API keys never in browser	All third-party API keys (OpenWeatherMap, Groq, Tavily, Twelve Data) are stored in Supabase Secrets and injected into the Deno Edge Function runtime at deploy time. The browser bundle contains only the Supabase anon key, which is public and RLS-scoped.
provider_refresh_token stripped from localStorage	The secureStorage adapter in src/lib/supabase.ts intercepts Supabase's default localStorage writes and removes provider_refresh_token before it is persisted. The short-lived provider_token lives in memory only. An XSS attacker reading localStorage cannot steal the refresh token.
Diary PIN: SHA-256, browser-side	setPIN() hashes the user's PIN using the Web Crypto API (SHA-256) before storage. verifyPIN() hashes the input and compares hashes client-side. The raw PIN is never stored or transmitted.
RLS at DB layer, not API layer	All user data tables enforce auth.uid() = user_id via Supabase RLS policies. Even if application-layer checks are bypassed by a bug, the database rejects unauthorized reads and writes.
JWT required for user-data Edge Functions	The events, tasks, fitness, and smart-widget Edge Functions require a valid Supabase JWT. The server validates with supabase.auth.getUser(jwt) and returns 401 on failure. Public Edge Functions (weather, stocks, tavily, groq) do not require JWT.
XSS localStorage audit	A full audit of all localStorage usage was completed and documented in docs/security/localStorage-audit.md (commit 86d7847). The audit identified provider_refresh_token as the only high-risk value; secureStorage was added as the fix.

Acknowledged limitation: the GCP OAuth app is in Testing mode. Users see an "unverified app" warning on first login. Full GCP verification requires separate Google review for Calendar/Tasks sensitive scopes and an annual CASA Tier 2 security assessment for the Fitness restricted scope.

9. Development Process

9.1 Sprint Summary

The project ran four one-week sprints from May 6 to May 27, 2026, with Jira as the sprint management tool. Tasks were migrated to GitHub Issues on 2026-05-23 for Milestone 4 compliance.

Sprint	Period	Planned	Done	Modified	Status
Sprint 1 — Foundation	May 6–11, 2026	3	3	0	Completed
Sprint 2 — Integration	May 12–13, 2026	3	3	0	Completed
Sprint 3 — Feature Completion	May 14–20, 2026	3	2	1	1 Modified (SCRUM-24)
Sprint 4 — Final Polish	May 21–27, 2026	3	3	0	Completed
Total	May 6–27, 2026	12	11	1	92% delivery rate

9.2 Schedule Changes

Three planned tasks were formally modified. Each change was made for a concrete reason and documented in SCHEDULE.md.

SCRUM-28: Vercel to Cloudflare Workers

Original plan: deploy to Vercel. Actual: deploy to Cloudflare Workers. The reason was SPA routing. Vercel's default behavior returns 404 for unknown paths, which breaks Google OAuth callbacks (`/?code=xxx`). Cloudflare Workers' wrangler.jsonc supports `not_found_handling: "single-page-application"`, which routes all paths to `index.html`. This was the correct call for an OAuth-dependent app.

SCRUM-16 / SCRUM-25: Persona Selector to 8-Category Interest Selection

Original plan: a dedicated Persona picker in onboarding. Actual: 8 fixed interest categories (news, tech, fashion, finance, health, food, entertainment, sports). We found during implementation that category-based selection produces stronger personalization signal: each selected category feeds directly into Tavily news queries and Groq context via `buildPersonaContext()`. A persona label produces weaker signal than explicit interest categories. The persona field and `setPersona()` remain in the codebase and can accept an explicit value if a future UI exposes one.

SCRUM-24: Desktop History to Smart Widget Keyword Learning + Briefing Snapshots

Original plan: integrate a browser history API for passive personalization. Actual: two replacement systems. Desktop browser history access requires a chrome.history Chrome Extension permission that is incompatible with an SPA deployed on Cloudflare Workers. The replacements collect equivalent signal from in-app activity: Smart Widget keyword learning from Q&A and diary text, and briefing snapshots for history context.

9.3 Team Contributions

All three team members contributed roughly equally. This section describes each member's planned work plus the additional items they shipped beyond the original sprint scope.

Kwon Dahyun — Frontend, UI/UX

Planned scope: Persona settings design and Diary Card wireframes (SCRUM-16), full Korean/English i18n across all components (SCRUM-20), modal portal implementation and diary feedback UI (SCRUM-23), and the Sprint 4 UI/UX audit (SCRUM-26).

Beyond scope: TypeScript migration across ~365 files (commit 365b392) — completed in a single session on 2026-05-24. Smart Widget category override button with emoji and dropdown. Drag-and-drop stock symbol reorder. 3-level calendar header drill-down (month / year / decade) with Today button fix. Font-size setting threaded into Diary, Event, and Task panels. News image broken fallback (Newspaper icon placeholder).

Choo Sungmin — Backend, UI/UX

Planned scope: PostgreSQL schema optimization and RLS policies (SCRUM-17), Diary Card response integration into the main diary (SCRUM-21), database indexing and RLS audit (SCRUM-27), and replacement of SCRUM-24 with Smart Widget keyword learning and briefing snapshots.

Beyond scope: Stocks widget overhaul including 8 fixed indices, strict ticker validation, and the unified modal split (fixed indices vs. user stocks). `api_cache` TTL extended from 1h to 6h. `personalizationService` with the 30-day decay model. Lazy diary synthesis for missed days (`diaryGenerationService.generateAndSaveDiaryForDate`).

Ahn Minsoo — Infrastructure, DevOps, Backend

Planned scope: Smart Widget implementation and Briefing sync (SCRUM-18), 30-day keyword decay formula and API caching (SCRUM-22), Persona-Groq bridge and error handling (SCRUM-25), and Cloudflare Workers deployment (SCRUM-28).

Beyond scope: Tavily Korean post-processing pipeline (`translateArticlesToKorean`, `buildLocalizedTrendTitles`). Groq `charset=utf-8` fix for Korean text. Bundle size reduction of

27.9% (1,025.94 kB to 739.50 kB). ARCHITECTURE.md and docs/security/localStorage-audit.md authored. useOnboardingStore extracted from useAuthStore/useSettingsStore.

9.4 Quality Process

Bug tracking ran through three stages:

7. CHANGELOG.md during development (Sprints 1-4): 27 dated fix rounds, each recording the date, affected files, root cause, fix, and follow-up items.
8. GitHub Issues migration on 2026-05-23 for Milestone 4 compliance. 9 retroactive issues filed and closed for transparency.
9. KNOWN_ISSUES.md as a staging area: 16 total items tracked, 10 resolved (including 4 fixed on 2026-06-04: Korean Smart Widget, Google Calendar sync, Stocks Edge Function deploy, Groq charset fix).

The team self-graded the project A- in the Milestone 4 Progress Update. The rationale: the cross-verification checklist in todo.md was only partially completed at submission time, approximately 10 GitHub issues remained open, and two Edge Function deploys required manual redeploy. All three items were resolved or substantially addressed by 2026-06-04.

10. API Design

The project documents 26 API endpoints across three categories: Supabase Edge Functions, Supabase REST (PostgREST), and Supabase Auth client calls. Full contracts are in Milestone/API.md. A structured reference spreadsheet is also available:

https://docs.google.com/spreadsheets/d/1s0TRC2M82RqDSFkEVq3LXFjo7rITSezO1_mNGqPBDIU/edit?usp=sharing.

10.1 Edge Functions (11 endpoints)

#	Function	Auth	Method	Purpose
1	weather	Public	POST	OpenWeatherMap current weather + AQI. Input: {lat, lon}. Returns temp, condition, humidity, precipitation, airQuality.
2	stocks	Public	POST	Stock quotes. Primary: TwelveData. Fallback: Yahoo Finance, Stooq CSV, ER-API. Returns type and currency metadata. 8 hardcoded internal symbols mapped to exchange tickers.
3	tavily (trends)	Public	POST	Real-time trend headlines. mode=trends. Returns trends[] array and article results.
4	tavily (news)	Public	POST	Personalized news. mode=news. Top-5 interest keywords prepended to query. 5 local + 5 global articles fetched in parallel.
5	tavily (search)	Public	POST	Generic search for Smart Widget sections. mode=search. Supports include_domains and exclude_domains filters.
6	groq	Public	POST	Groq LLM completion. Input: {system, prompt, model?, temperature?}. Returns {text}. Content-Type: application/json; charset=utf-8 to prevent Korean garbling.
7	events	JWT + Google	POST	Google Calendar CRUD. action: list / create / update / delete / read. Uses explicit timeMin/timeMax for timezone-correct event fetching.
8	tasks	JWT + Google	POST	Google Tasks CRUD. action: list / create / update / delete / move / clearCompleted.
9	fitness	JWT + Google	POST	Google Fit: steps, sleep, calories, heart rate.
10	smart-widget	JWT	POST	Personalized Smart Widget content. Input: {keyword, persona (nested object), token?, lat, lon}. 7-route dispatcher (weather/stocks/calendar/fitness/trends/restaurants/n one). Step 0: Groq route classification. Steps 1–2:

				research planning + Tavily search. Output: {keyword, emoji, lastUpdated, sections[], debug}.
11	google-refresh	Public	POST	Google OAuth token refresh. Input: {refresh_token}. Returns {access_token, expires_in}. Final fallback in useAuthStore.ensureProviderToken() 4-step chain. Secrets (GOOGLE_CLIENT_ID, GOOGLE_CLIENT_SECRET) held server-side.

10.2 Supabase REST Endpoints (13 endpoints)

#	Table	Methods	Purpose
12-13	api_cache	GET, POST	Read cached API responses (6-hour TTL check). Write/upsert with semantic string PK.
14-15	user_settings	GET, POST	Read and upsert all user preferences and personalization data.
16-17	widget_layouts	GET, POST	Read and upsert widget grid layout per breakpoint.
18	todos	GET/POST/PATCH/DELETE	Full CRUD for daily todos. is_fixed flag controls midnight reset behavior.
19	smart_keywords	GET/POST/DELETE	Manage user Smart Widget keyword list. Unique constraint on (user_id, keyword).
20	diaries	GET/POST/PATCH	Read and write diary entries per date. Stores aiGeneratedDiary, editedDiary, feedback history.
21	user_qa	GET, POST	Daily Q&A question/answer pairs. Indexed on (user_id, asked_date).
22	briefing_snapshots	GET, POST	Time-stamped briefing snapshots. Used by diaryGenerationService for missed-day recovery.
23	keyword_score_log	POST, DELETE	Insert keyword scoring events; delete logs older than 30 days during nightly batch.

10.3 Auth Client Calls (3 endpoints)

#	Method	Purpose
24	supabase.auth.signInWithOAuth()	Initiates Google OAuth flow with calendar, tasks, and fitness scopes.
25	supabase.auth.getSession() / refreshSession()	Loads existing session on page load. Used in App.tsx auth bootstrap.
26	supabase.auth.signOut()	Clears session, resets all Zustand stores, removes mb_login from localStorage.

10.4 Cross-Cutting Concerns

Error Handling

All Edge Function call sites use `handleApiError(err, context)` from `src/utils/errorHandler.ts`. On failure, an `ApiError` is thrown with a `userVisible` flag. Catch blocks set `apiStatus="error"` per widget and fall back to mock data from `src/mock/data.ts`. The 25-second timeout on all Edge Function calls prevents indefinite loading states.

Auth

Public Edge Functions (weather, stocks, tavily, groq, google-refresh) require only the Supabase anon key. User-data Edge Functions (events, tasks, fitness, smart-widget) require a valid JWT from `supabase.auth.getSession()`.

11. Testing

The project does not have an automated test suite. All verification is manual. This is an acknowledged gap noted in the self-assessment.

11.1 Manual Verification Checklist

The README.md §Testing section defines a 14-point verification checklist:

#	Test Case
1	Login flow: Google OAuth consent, redirect back, dashboard renders
2	Widget rendering: all default widgets appear within 3 seconds of login
3	Last-updated time: all widget headers show "방금" or less than 2 minutes after login, not a stale timestamp from a previous session
4	Manual refresh: clicking the refresh icon on any widget triggers a force=true fetch and shows a loading indicator
5	Language toggle: switching language reloads news, trends, and Smart Widget content in the selected language
6	Settings persistence: changing theme, temperature unit, or stock symbols persists across page reload
7	Calendar: add an event via EventPanel; event appears on calendar. Year picker and decade view function; Today button is visible from all header levels.
8	News images: broken image URLs show a Newspaper icon placeholder with no blank gap in the layout
9	Smart Widget: category button shows [category emoji + dropdown arrow]; clicking opens dropdown; pencil icon edits the keyword
10	Diary: answer today's Q&A, save, reload; answer is retained. PIN gating works correctly.
11	AI Briefing: briefing generates; clicking opens detail modal; all sections (header / schedule / yesterday / latest_info) render correctly
12	News image fallback: force a broken image URL in DevTools; Newspaper icon placeholder appears with no blank gap
13	Login timestamp reset: log out, wait, log back in; widget last-updated labels show current timestamps, not stale ones
14	Smart Widget category override: click the [category emoji + dropdown] button; select a different category; override applies and content reloads

11.2 Cross-Verification

The team's process requires each completed feature to be verified by a team member who did not implement it. Bugs found during cross-verification are filed as GitHub Issues. The

cross-verification pass was partially completed as of June 2026; remaining items were tracked in todo.md.

11.3 Future Testing Work

Adding an automated test suite is the first item on the future work list. Candidate areas: unit tests for personalizationService (score calculation, decay formula), integration tests for the caching layer (TTL logic, fingerprint generation), and end-to-end tests for the login flow and briefing generation.

12. Known Issues and Limitations

This section lists known issues at the time of submission. All items are tracked in GitHub Issues.

GH #	Issue	Severity	Notes
#2	Diary feedback rewrite not applying	Major	pendingRewrite not propagating to UI or Groq call failing silently.
#4	Briefing latest_info shows generic definitions instead of personalized content	Major	smartSummaries from useWidgetStore may not be passed into generateDetailedBriefing context.
#5	Briefing shows "No keyword info collected yet" indefinitely	Minor	Smart Widget keyword population timing or empty state not communicated clearly in UI.
#9	Google Fitness live API sync: Edge Function deployed but live token connection missing	Major	Fitness scope requires full GCP verification. Edge Function is implemented and deployed.
#10	Voice feature: state exists but no UI or TTS	Minor	voiceOn field in useSettingsStore. No TTS implementation. Future work.
#11	NewsDetailModal file exists but is unused	Trivial	Replaced by direct URL navigation. Should be removed in a future cleanup.
#12	Trends widget: no detail view or pagination	Minor	News and stocks "view all" modals implemented. Trends left/right pagination not implemented.
#13	Diary PIN setup and recovery UI not exposed in Settings	Major	PIN hashing, storage, and verification are implemented. The Settings UI for set/modify/recovery flows needs to be built.

GCP OAuth App in Testing Mode

The app remains in GCP Testing mode. All users see an "unverified app" warning on first login. The workaround is to click Advanced then Continue. In Testing mode, OAuth tokens may expire after 7 days. Full verification requires separate Google review for Calendar/Tasks sensitive scopes and an annual CASA Tier 2 security assessment for the Fitness restricted scope.

No Automated Test Suite

All testing is manual (14-point checklist). There are no unit, integration, or end-to-end tests. This is acknowledged as a process gap and is the first item on the future work list.

13. Reflection and Future Work

13.1 What Worked

Supabase as BaaS

Choosing Supabase as the backend platform was the right call for a three-person team. Auth, database, and serverless functions are all managed in one place. RLS was particularly valuable: the team could iterate on features without worrying about access control because the database enforced it automatically.

Groq Speed

Near-real-time LLM inference made the AI briefing feel like a native feature rather than a loading screen. At temperature=0.1, output is consistent enough that the deterministic-sections pattern works reliably. If we had used a slower model, the perceived responsiveness of the briefing and diary generation would have required more careful engineering to hide latency.

Zustand Store-per-Domain

The 10-store architecture scaled well across four sprints. When features were added (useOnboardingStore extraction, useBriefingHistoryStore for snapshots), the change was localized to one store file and one consumer component. There were no cross-store state bugs that required global refactoring.

13.2 What Was Hard

Google OAuth Token Lifecycle

The provider_token expiry problem took multiple iterations to solve correctly. The first attempt was to force re-login when the token expired, which was bad UX. The second attempt stored the refresh token in localStorage, which was a security risk. The final solution — the google-refresh Edge Function with invokeGoogleWithAuth auto-retry — recovers expired sessions transparently without storing the refresh token in localStorage.

Korean Article Quality

Getting Tavily to return high-quality Korean articles required multiple layers of filtering. A simple include_domains list was not enough — some Korean news domains return machine-translated English content or low-quality articles. The final solution uses three layers: domain allowlist, scoreArticleForLanguage() Hangul detection, and a Groq post-processing step (translateArticlesToKorean()) that backfills translated content into api_cache.

TypeScript Migration Mid-Sprint

The full TypeScript migration (~365 files, commit 365b392) happened mid-project during Sprint 2, completed in a single day (2026-05-24). Using `strict:false` reduced the migration effort substantially — strict mode would have required annotating every function parameter and return type. `strict:false` gives build-time safety benefits without requiring full annotation coverage.

13.3 Future Work

Area	Item
Testing	Automated test suite: unit tests for <code>personalizationService</code> , integration tests for the 3-layer cache, end-to-end tests for login and briefing generation.
Auth	Full GCP OAuth app verification (Calendar/Tasks + Fitness scopes). Removes the "unverified app" warning for all users.
Voice	TTS implementation using the <code>voiceOn</code> state that already exists in <code>useSettingsStore</code> . Would allow the briefing to be read aloud hands-free.
Mobile / PWA	The current layout is desktop-first. A responsive redesign and PWA manifest would make the app usable on mobile.
Trends Detail View	The Trends widget shows top items with an expand/collapse toggle. A full detail view with pagination was not implemented.
Diary PIN Settings UI	The PIN hashing, storage, and verification are fully implemented. The Settings UI for set/modify/recovery flows needs to be built and exposed to users.
Chrome Extension (Hybrid Build)	The <code>npm run build:extension</code> script exists but the hybrid extension variant was deferred to v2. Publishing to the Chrome Web Store would make new-tab replacement seamless.